

Meeting Cost Meter — Technical Design Document

Prepared for the Zoom App Marketplace submission of **Meeting Cost Meter** (Production), by Transformative Leadership Lab LLC.

1. App overview

Meeting Cost Meter is an in-client Zoom App that displays the live, running dollar cost of a meeting. The presenter enters a manual attendee count and a single hourly opportunity-cost rate; the app computes a running total (`attendees × rate × elapsed time`) and renders it either in the app's side panel or as an overlay composited onto the presenter's own camera video via the Zoom Layers API. It exists to make the cost of meeting time visible to participants.

The app is deliberately minimal: a single attendee-count-times-rate model, no per-participant data, and no server-side storage of any kind.

2. App type and surfaces

- **Type:** Zoom App (in-client, built on the Zoom Apps SDK / `@zoom/appssdk`).
- **Management model:** User-managed. Individual users install it for their own use.
- **Surfaces:** in-meeting **side panel** (running context `inMeeting`) and **camera overlay** (running context `inCamera` , Zoom Layers API).
- **Client scope:** `zoomapp:inmeeting` only. No REST API scopes are requested.

3. Architecture and components

Component	Technology	Role
Client	React + Vite (static bundle)	The in-client UI; talks to Zoom via <code>@zoom/appssdk</code> (<code>postMessage</code>). Computes and renders the cost.
Server	Node.js 22 + Express on Railway	Serves the built client bundle and a small set of endpoints (below). Holds no database.
Public site	Static HTML on GitHub Pages	Legal + support pages (privacy, terms, support, documentation). No JS, no external requests.

A visual architecture & data-flow diagram accompanies this submission (`meeting-cost-architecture.png`), also published at <https://thomasbcox.github.io/zoom-meeting-cost/meeting-cost-architecture.svg> .

Server endpoints: - `GET /` and static assets — serves the built React client. - `GET /api/health` — liveness check (`{ ok, zoomConfigured }`). - `GET /api/version` — deployed environment, git commit, boot time (non-sensitive). - `POST /api/log` — client diagnostics sink. Writes a single compact log line to stdout/stderr. **Stores nothing.** Payloads are minimized at the source (no PII). - `GET /auth/install`, `GET /auth/callback` — Zoom OAuth 2.0 authorization-code flow. - `POST /auth/deauthorize` — Zoom deauthorization webhook (HMAC-verified; see §9).

4. Data flow

1. A user installs the app via Zoom OAuth 2.0 (authorization-code grant). The server exchanges the code for a token over `https://zoom.us/oauth/token` (HTTP Basic with client id/secret). The token is **not persisted**.
2. In a meeting, the app loads in the Zoom client from the Railway-served bundle. It calls the Zoom Apps SDK for its running context and the presenter's own camera/identity context (to place the overlay on the presenter's video).
3. The presenter enters an **attendee count** and an **hourly rate** in the side panel. These values live only in browser memory for the session.
4. The client computes the running cost locally and, in overlay mode, pushes a **sanitized aggregate display state** from the panel to the camera instance via the SDK's `postMessage`. That message contains only: `{ status, totalCost, costPerSecond, elapsedSeconds, attendees, currency, updatedAt, displayIntervalSeconds, prefs:{} }` — no per-person data, no meeting content (`displayIntervalSeconds` is a non-identifying scalar: how often the displayed number steps).
5. Diagnostics/errors may be POSTed to `/api/log` (shape-only, no PII) and are written to ephemeral platform logs. Nothing is stored.

No participant list is ever read — the attendee count is a manual input. The participant-read scopes were intentionally removed.

5. Data collected, stored, and retention

- **Collected from the user:** attendee count, one hourly rate, display cadence. Held in **browser memory only**, for the duration of the meeting; discarded when the meeting ends. Never transmitted to or stored by the server.
- **Stored server-side: none.** There is no database, no key-value store, no file persistence of user data. (An earlier encrypted rate store was removed; the app is session-only by design.)
- **OAuth token:** used transiently for the token exchange; not persisted.
- **Diagnostics logs:** `/api/log` writes minimized diagnostics to the platform's ephemeral stdout/stderr. Payloads are minimized **at the source** — the in-Zoom probe transmits only the *shape* of Zoom data (field names/counts, never values), and error reports carry a fixed allowlist of

technical fields (error text/stack, source file/line, request path, user agent). Because the endpoint records the body the client sends, this is a data-minimization control, not an absolute guarantee: it is not intentionally populated with participant PII or the presenter's figures. (This mirrors the Privacy Policy's wording.)

- **Retention:** because nothing is stored, there is nothing to retain or expire.

6. OAuth scopes and justification

Scope	Why it is needed
zoomapp:inmeeting	Run the app within the Zoom Meeting client (side panel + camera overlay). This is the app's sole capability.

No REST API scopes are requested. The app does not call the Zoom REST API, does not read participant or user directory data, and does not access recordings, chat, or meeting content. This is the minimum scope set for an in-meeting-only app.

7. Authentication and authorization

- **Install/authorization:** Zoom OAuth 2.0 authorization-code grant. Redirect URI is allow-listed in the Marketplace config and pinned via Strict Mode (exact-match). Because the app makes no Zoom REST API calls, the resulting access token is **not used or stored** after the exchange — the flow exists only to install and authorize the in-client app.
- **In-client:** the Zoom Apps SDK supplies the running context; there is no separate application login and no user account system.
- **No elevated/admin access:** user-managed app; access is limited to the installing user's own in-meeting context.

8. Third-party services / subprocessors

Service	Purpose	Data shared
Zoom	Host client + SDK (postMessage) + OAuth	OAuth code/token exchange only; SDK calls stay on-device via postMessage
Railway	Application hosting (server + client bundle)	Serves static assets + endpoints; no user data stored
GitHub Pages	Static legal/support site	None (static pages, no user input)

There is **no analytics provider, no external CDN, no tracker, and no third-party data sharing**. The Content-Security-Policy pins `connect-src 'self'`, so the client makes no cross-origin network calls other than its own same-origin `/api/log`.

9. Security controls

- **Transport:** HTTPS everywhere (Railway TLS). `upgrade-insecure-requests` in CSP.
- **OWASP secure headers on every response:**
- **Content-Security-Policy (tightened):** `default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self' data:; connect-src 'self'; object-src 'none'; frame-src 'none'; worker-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'self' https://*.zoom.us https://*.zoom.com; upgrade-insecure-requests`
- `Strict-Transport-Security: max-age=31536000; includeSubDomains`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: same-origin`
- **OAuth:** client secret sent only server-side (HTTP Basic to Zoom's token endpoint); redirect URIs allow-listed + Strict Mode.
- **Deauthorization webhook:** verifies Zoom's HMAC signature (`x-zm-signature = v0= HMAC-SHA256 over v0:<timestamp>:<raw body>`) with a dedicated webhook **secret token** (separate from the OAuth client secret); rejects replays outside a ± 5 -minute window; uses constant-time comparison; the rate limiter + raw-body capture run before the JSON parser so malformed/oversized floods are bounded. Answers the `endpoint.url_validation` handshake. Because no user data is stored, the purge is a no-op — verify and acknowledge 200.
- **Diagnostics minimization:** `/api/log` payloads are minimized at the source (shape-only probe; fixed technical-field allowlist for errors). Because the endpoint logs the submitted body, this is a minimization control rather than an absolute guarantee — consistent with the Privacy Policy.
- **Secure development lifecycle:** every change ships via reviewed pull request on a protected `main` branch with a passing test suite; static analysis via GitHub CodeQL; dependency monitoring via Dependabot. (Matches the public Security page.)
- **Secret handling:** credentials live only in platform environment variables; a repository secret-scanning pre-commit hook blocks accidental commits.
- **Excluded by Zoom policy:** no load testing / DoS simulation is expected in review.

10. Deauthorization and data deletion

On app uninstall, Zoom POSTs an `app_deauthorized` event to `POST /auth/deauthorize`. The endpoint verifies the signature, performs the data purge (a **no-op**, because no per-user data is stored), and

returns `200` . Users can uninstall at any time from the Zoom App Marketplace / their Zoom account; there is no stored data to delete and no separate deletion request is necessary.

11. Infrastructure and hosting

- **Hosting:** Railway (Node 22 / Express). Two environments (Development, Production), each with its own Zoom credential block and domain; auto-deploy from the `main` branch.
- **Production domain:** <https://zoom-meeting-cost-production.up.railway.app> — a Railway-hosted domain controlled solely by us, serving only this app's static client bundle and its first-party endpoints (health, diagnostics log, OAuth callback, deauthorization webhook). No third party can publish or control content on it.
- **Static site:** GitHub Pages at <https://thomasbcox.github.io/zoom-meeting-cost/> .